

Rabin-Karp Algorithm

Rabin-Karp algorithm is an algorithm used for searching/matching patterns in the text using a hash function. Unlike Naive string matching algorithm, it does not travel through every character in the initial phase rather it filters the characters that do not match and then performs the comparison.

A hash function is a tool to map a larger input value to a smaller output value. This output value is called the hash value.

How Rabin-Karp Algorithm Works?

A sequence of characters is taken and checked for the possibility of the presence of the required string. If the possibility is found then, character matching is performed.

Let us understand the algorithm with the following steps:

1. Let the text be:

A	B	C	C	D	D	A	E	F	G
---	---	---	---	---	---	---	---	---	---

Text

And the string to be searched in the above text be:



Pattern

2. Let us assign a `numerical value(v)/weight` for the characters we will be using in the problem. Here, we have taken first ten alphabets only (i.e. A to J).



Text Weights

3. `n` be the length of the pattern and `m` be the length of the text. Here,
`m = 10` and `n = 3`.

Let `d` be the number of characters in the input set. Here, we have taken input set {A, B, C, ..., J}. So, `d = 10`. You can assume any suitable value for `d`.

4. Let us calculate the hash value of the pattern.



hash value = 6

$$\begin{aligned}
 \text{hash value for pattern}(p) &= \sum(v * d_{m-1}) \bmod 13 \\
 &= ((3 * 10^2) + (4 * 10^1) + (4 * 10^0)) \bmod 13 \\
 &= 344 \bmod 13 \\
 &= 6
 \end{aligned}$$

In the calculation above, choose a prime number (here, 13) in such a way that we can perform all the calculations with single-precision arithmetic.

The reason for calculating the modulus is given [below](#).

5. Calculate the hash value for the text-window of size .

$$\begin{aligned}
 &\text{For the first window ABC,} \\
 \text{hash value for text}(t) &= \sum(v * d_{n-1}) \bmod 13 \\
 &= ((1 * 10^2) + (2 * 10^1) + (3 * 10^0)) \bmod 13 \\
 &= 123 \bmod 13 \\
 &= 6
 \end{aligned}$$

6. Compare the hash value of the pattern with the hash value of the text. If they match then, character-matching is performed.

In the above examples, the hash value of the first window (i.e.) matches with so, go for character matching between ABC and CDD. Since they do not match so, go for the next window.

7. We calculate the hash value of the next window by subtracting the first term and adding the next term as shown below.

$$\begin{aligned}
 t &= ((1 * 10^2) + ((2 * 10^1) + (3 * 10^0)) * 10 + (3 * 10^0)) \bmod 13 \\
 &= 233 \bmod 13 \\
 &= 12
 \end{aligned}$$

In order to optimize this process, we make use of the previous hash value in the following way.

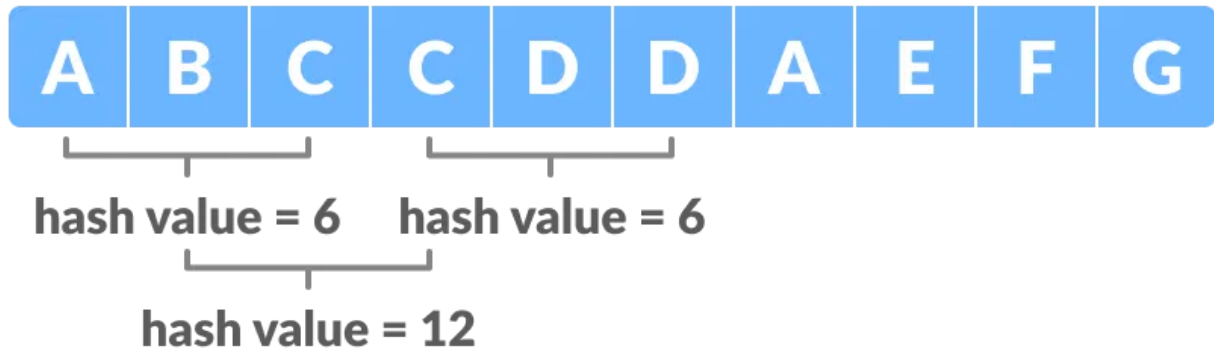
$$\begin{aligned}
 t &= ((d * (t - v[\text{character to be removed}] * h) + v[\text{character to be added}]) \bmod 13 \\
 &= ((10 * (6 - 1 * 9) + 3) \bmod 13
 \end{aligned}$$

= 12

Where, $h = d^{m-1} = 10^{3-1} = 100$.

8. For BCC, $t = 12$ ($\neq 6$). Therefore, go for the next window.

After a few searches, we will get the match for the window **CDA** in the text.



Hash value of different windows

Algorithm

```
n = t.length
m = p.length
h = dm-1 mod q
p = 0
t0 = 0
for i = 1 to m
    p = (dp + p[i]) mod q
    t0 = (dt0 + t[i]) mod q
for s = 0 to n - m
    if p = ts
        if p[1.....m] = t[s + 1..... s + m]
            print "pattern found at position" s
    If s < n-m
        ts + 1 = (d (ts - t[s + 1]h) + t[s + m + 1]) mod q
```

Python, Java and C/C++ Examples

Python

Java

C

C++

```
// Rabin-Karp algorithm in C++

#include <string.h>

#include <iostream>
using namespace std;

#define d 10

void rabinKarp(char pattern[], char text[], int q) {
    int m = strlen(pattern);
    int n = strlen(text);
    int i, j;
    int p = 0;
    int t = 0;
    int h = 1;

    for (i = 0; i < m - 1; i++)
        h = (h * d) % q;

    // Calculate hash value for pattern and text
    for (i = 0; i < m; i++) {
        p = (d * p + pattern[i]) % q;
        t = (d * t + text[i]) % q;
    }

    // Find the match
    for (i = 0; i <= n - m; i++) {
        if (p == t) {
```

Python, Java and C/C++ Examples

PythonJavaCC++

```
}

// Find the match
for (i = 0; i <= n - m; i++) {
    if (p == t) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }

        if (j == m)
            cout << "Pattern is found at position: " << i + 1 << endl;
    }

    if (i < n - m) {
        t = (d * (t - text[i] * h) + text[i + m]) % q;

        if (t < 0)
            t = (t + q);
    }
}

int main() {
    char text[] = "ABCCDDAEFG";
    char pattern[] = "CDD";
    int q = 13;
    rabinKarp(pattern, text, q);
}
```

Limitations of Rabin-Karp Algorithm

Spurious Hit

When the hash value of the pattern matches with the hash value of a window of the text but the window is not the actual pattern then it is called a spurious hit.

Spurious hit increases the time complexity of the algorithm. In order to minimize spurious hit, we use modulus. It greatly reduces the spurious hit.

Rabin-Karp Algorithm Complexity

The average case and best case complexity of Rabin-Karp algorithm is $O(m + n)$ and the worst case complexity is $O(mn)$.

The worst-case complexity occurs when spurious hits occur a number for all the windows.

Rabin-Karp Algorithm Applications

- For pattern matching
 - For searching string in a bigger text
-

Previous Tutorial:
[Backtracking Algorithm](#)

Did you find this article helpful?



Your builder path starts here.

Escape tutorial hell and ship real projects.

Try Programiz PRO